

Certificación Firmware V5.0 Rechazada

Usuario: [File: PROMPT_AUDITORIA_RONDA7_CERTIFICACION_V5.md]

use.ai: [RECHAZADO] para certificar el cierre del Subhito 2.2 con el firmware V5.0 presentado. La validación por ancho de nivel es una mejora razonable para ensayar con el hardware congelado, pero el código y las evidencias disponibles no demuestran inmunidad EMI ni aptitud metrológica en el rango de permeado.

El rechazo corresponde a la **certificación**, no a la continuidad del proyecto: pueden avanzar sin desoldar componentes, corrigiendo el software y realizando pruebas de aceptación. Hay, además, una contradicción de pines que debe resolverse antes de energizar el banco.

Alcance de esta revisión: auditoría documental y análisis estático. No he compilado el proyecto ni ensayado el hardware. Por ello, el código que entrego es una implementación candidata para validación, no un firmware certificado ni una declaración de seguridad de la planta. El `.ino` recibido es un extracto: faltan el controlador de bomba, las estructuras de registro, los manejadores web y otras dependencias para revisar y entregar honestamente la aplicación completa.

1. Hallazgos que impiden aprobar

1.1. Contradicción crítica en el conexionado

La descripción y `config.h` asignan funciones diferentes a los mismos GPIO:

Función	Descripción del hardware	<code>config.h</code>
Sensor alimentación	GPIO 18	GPIO 14
Sensor permeado	GPIO 19	GPIO 27
PUL / DIR del motor	No especificados allí	GPIO 18 / GPIO 19

Debe verificarse el cableado real. Si los sensores están efectivamente en 18 y 19, el firmware está leyendo otros pines y asignando los primeros al motor. Esto puede explicar señales incorrectas sin recurrir exclusivamente a una hipótesis de EMI.

No propongo cambiar el hardware: propongo que el software represente fielmente las conexiones existentes.

1.2. El rango de permeado no está metrológicamente demostrado

La ficha de DFRobot para YF-S401 especifica **0,3 a 6 L/min**, con exactitud declarada en 0,3 a 3 L/min y 5880 pulsos/L. Los 10 a 25 mL/min pretendidos están entre 12 y 30 veces por debajo de ese límite inferior. Incluso 200 mL/min queda por debajo. [DFRobot](#)

También existen fichas publicadas bajo la misma denominación con rangos distintos, por lo que debe identificarse la **variante realmente instalada**. [Ficha YF-S401](#)

Reducir `qMin_mLmin` a 4 no reduce la fricción del rotor ni demuestra que gire de forma reproducible a ese caudal. Si el rotor permanece detenido con flujo real, ningún filtro digital puede reconstruir el volumen que no produjo pulsos.

Consecuencia para la tesis: el sensor puede investigarse fuera de su rango publicado, pero ese uso necesita caracterización experimental propia. No debe presentarse como rango ya validado.

1.3. El filtro pierde continuidad entre ventanas

En la ISR:

```
if (_validosTotal == 0 || periodo >= _p.periodoMin_us)
```

Pero `actualizar()` reinicia `_validosTotal` cada segundo. Por tanto, el **primer pulso de cada ventana queda exento del control de período mínimo**, aunque el pulso anterior haya ocurrido inmediatamente antes del corte.

El estado temporal permanente debe estar separado de los contadores de ventana.

1.4. La estimación a baja frecuencia no es recíproca

Con un pulso en una ventana, el código impone:

```
_f = 1.0f / dt_s;
```

Con ninguno, impone cero. Esto produce lecturas alternantes para señales lentas, aunque los pulsos reales sean perfectamente periódicos.

Para un factor nominal de 98:

$$f(4\ \mathrm{mL/min}) = 98 \frac{4}{1000} = 0.392\ \mathrm{Hz}$$

$$T = 2.55\ \mathrm{s}$$

El tiempo de espera de 1,5 s y el corte adicional `_f < 0.5f` son incompatibles con esa sensibilidad pretendida. Esta observación no demuestra que el sensor mida 4 mL/min; demuestra que **el algoritmo tampoco permitiría hacerlo correctamente** si llegara a generar esos pulsos.

1.5. El volumen se condiciona indebidamente al caudal mostrado

La expresión:

$$V_L = \frac{N}{60K}$$

es dimensionalmente correcta si K está expresado en Hz/(L/min). El problema está en acumularla solamente cuando el caudal supera los cortes y la compuerta de bomba.

Esto descarta pulsos previamente aceptados por razones de presentación o de plausibilidad. Deben separarse:

Conteo geométricamente aceptado: resultado del procesamiento digital.

Volumen estimado: conversión de ese conteo mediante una calibración identificada.

Validez metrológica: resultado de los ensayos, el rango y las condiciones de adquisición.

Un dato sospechoso debe conservarse y marcarse, no desaparecer silenciosamente.

1.6. Bomba apagada no equivale necesariamente a caudal nulo

Puede existir flujo por **presión residual, desnivel hidrostático, descarga de elasticidad del circuito o circulación externa**. El estado del motor no basta para imponer $Q_{perm} = 0$.

La compuerta propuesta también oculta justamente el fenómeno que se quiere investigar: pulsos con la bomba parada. Debe convertirse en una bandera diagnóstica, sin alterar la medición.

1.7. La seguridad de presión no está implementada

Las constantes `TMP_MAX_SEGURA_BAR` y `RPM_ALARMA_MEMBRANA` **no constituyen protección de presión**. Sin medir presión, el software no demuestra que se respete el límite de 0,50 bar; una bomba de desplazamiento positivo puede elevarla ante una restricción incluso a bajas RPM.

Esta revisión no habilita operar la membrana sin una protección independiente adecuada y un procedimiento supervisado.

2. Inmunidad EMI con el hardware congelado

2.1. Corrección del cálculo RC

Para una carga exponencial ideal:

$$t_{10-90} = \tau \ln(9)$$

Con $\tau = 0.94 \text{ ms}$:

$$t_{10-90} \approx 2.07 \text{ ms}$$

No es 1,03 ms. Además, ese modelo describe principalmente la subida a través del pull-up. La bajada depende del transistor de salida y de las impedancias efectivas.

Tampoco corresponde usar el argumento sobre ausencia de Schmitt como hecho demostrado sin verificar las características de la entrada concreta. El punto comprobable es que **una transición lenta puede aumentar la sensibilidad a perturbaciones alrededor del umbral**.

2.2. Qué puede rechazar el filtro

El filtro de ancho puede rechazar pulsos digitales aislados si sus duraciones observadas están por debajo del umbral elegido. Es una estrategia **viable como hipótesis de trabajo** con el hardware actual.

Sin embargo, no demuestra rechazo universal:

El período del chopper no fija la duración del ruido en el GPIO. Que exista conmutación a 20 kHz no implica que toda perturbación registrada dure menos de 50 μs.

Las ráfagas pueden formar niveles largos. Una perturbación que produzca HIGH y LOW suficientemente largos puede superar ambos umbrales.

Los rebotes pueden eliminar pulsos reales. Un glitch HIGH breve dentro de un LOW verdadero puede invalidar la siguiente bajada y fragmentar el pulso.

Las interrupciones no son un registrador perfecto de flancos. Una ráfaga rápida puede producir eventos no atendidos individualmente.

Un tren falso indistinguible del verdadero es indistinguible por software. No existe una decisión fiable basada exclusivamente en esos mismos tiempos digitales.

Los valores de 200 y 600 μs son **parámetros candidatos**, no límites mecánicos demostrados. Necesitan contrastarse con las distribuciones de anchos observadas en líquido, en seco y durante perturbaciones.

2.3. Carga de interrupciones y memoria

`IRAM_ATTR` sobre dos funciones no basta para declarar toda la adquisición segura durante operaciones de flash. Deben verificarse el registro de la interrupción, las funciones llamadas y la ubicación de los datos. Espressif exige que una ISR declarada IRAM-safe acceda solamente a código y datos disponibles con la caché deshabilitada. [Espressif: asignación de interrupciones](#)

Conclusión: puede mantenerse el hardware mientras se ensaya la solución. No se puede garantizar con lo presentado la erradicación del 100% del EMI ni recuperar señales físicas que el acondicionamiento o el sensor no permiten distinguir.

3. Datalogger, balance hidráulico y Darcy

3.1. Diez segundos son aceptables para tendencias, no para cualquier objetivo

El intervalo de 10 s puede servir para seguimiento lento y ensayos escalonados. No caracteriza por sí solo la pulsación de una bomba de cuatro rodillos:

$$f_{\mathrm{rodillos}} = \frac{4, \text{RPM}}{60}$$

En el rango de 15 a 100 RPM resulta aproximadamente **1 a 6,67 Hz**. Guardar una lectura puntual cada 10 s puede introducir aliasing y ocultar transitorios.

Para el registro de tesis recomiendo **volumen por conteo durante el intervalo**, duración real del intervalo y caudal medio:

$$\overline{Q}_{\mathrm{mL/min}}$$

$$\frac{\Delta N}{60K} \frac{60,000}{\Delta t_s}$$

$$\frac{1000 \cdot \Delta N}{K \cdot \Delta t_s} \text{ s}$$

Conservar también RPM, conteos rechazados, modo de ensayo, versión de calibración y banderas de calidad.

3.2. La capacidad está sobreafirmada

Con 600 registros incluyendo una fila en cero:

$$t_{\mathrm{\acute{u}ltimo}} = (600 - 1) \times 10 = 5990 \text{ s}$$

Esto equivale a **99 min 50 s**, no a una muestra final a los 100 minutos. Para registrar desde 0 hasta 6000 s inclusive hacen falta 601 filas, sin contar posibles eventos adicionales.

Cambiar el intervalo tampoco evita por sí mismo la sobrescritura. Debe definirse explícitamente qué ocurre al llenarse el almacenamiento.

3.3. El extracto no demuestra el registro inicial

No se muestra `iniciarNuevoEnsayo()`, por lo que no es verificable que guarde una fila inicial ni que reinicie el calendario.

Una fila en $t=0$ debe ser un **evento de inicio**. No debe presentarse como caudal promedio de una ventana aún inexistente. Si se registra un valor previo, debe identificarse como tal.

3.4. Darcy: mejora correcta, representación incompleta

Suspender el cálculo con presión inventada es **correcto y obligatorio**.

Pero `TMP = 0.00` puede interpretarse como una presión medida igual a cero. Conviene representar:

```
TMP_bar = campo vacío
Darcy_valido = false
motivo = PRESION_NO_INSTRUMENTADA
```

En JSON debe emplearse `null`, no el token `NaN`, que no es JSON válido.

3.5. El balance debe conservar las discrepancias

No recomiendo:

```
qRet_mLmin = fmaxf(0.0f, qAlim - qPerm);
```

Para análisis conviene conservar **la diferencia con signo** y marcar inconsistencias. Recortar negativos borra evidencia de desajustes, desfases y errores de medición.

Además:

$$Q_{\mathrm{alim}} - Q_{\mathrm{perm}} - Q_{\mathrm{ret}}$$

$$\frac{dV_{\mathrm{sistema}}}{dt} \approx 0$$

La igualdad simplificada exige acumulación despreciable y ausencia de otras entradas o salidas. Con sensores pulsátiles y filtrados diferentes no debe interpretarse como balance instantáneo exacto.

4. Factores K: qué debe esperar Antonella

No debe fijarse anticipadamente ni 98 ni 196,5. Debe recalibrarse cada sensor después de cambiar el conteo, con el acondicionamiento definitivo.

La relación nominal publicada de 5880 pulsos/L corresponde a:

$$K = \frac{5880}{60} = 98 \text{ Hz/(L/min)}$$

Esa equivalencia está respaldada para la variante documentada, no necesariamente para cualquier unidad comercial etiquetada YF-S401. [DFRobot](#)

Si antes se contaban **exactamente dos eventos por pulso físico**, al corregirlos sería esperable que el factor se redujera aproximadamente a la mitad:

$$196.5/2 = 98.25$$

Pero eso es una **predicción condicional**, no un diagnóstico confirmado. También pueden intervenir variante del sensor, unidades, instalación, pulsación, errores de volumen de referencia o selección de flancos.

No hay evidencia para atribuir una duplicación exacta a la hidrodinámica del cartucho. Asimismo, $K_{\text{PERMEADO}} = 687.33$ merece investigación: es aproximadamente siete veces el nominal citado.

Para una recolección completa de $V_{\mathrm{ref},L}$ y N pulsos aceptados:

$$K = \frac{N}{60V_{\mathrm{ref},L}}$$

La calibración debe comprobar **repetibilidad, sesgo y dependencia con el caudal**. En el permeado, primero debe verificarse que exista una relación reproducible entre volumen y conteo; ajustar un factor no resuelve el bloqueo intermitente del rotor.

5. Código completo del módulo de adquisición propuesto

Entrego completos `caudalimetro.h` y `caudalimetro.cpp`, más un módulo de registro e instrucciones concretas de integración. **No los presento como compilados ni validados en banco.** Su destino es Arduino-ESP32; en ESP-IDF requieren Arduino como componente, no son código ESP-IDF puro.

La implementación mantiene el método de ancho de nivel para poder compararlo con V5. No introduce una supuesta eliminación universal del rebote.

5.1. Decisiones de diseño

Las modificaciones principales son:

Continuidad temporal: el último pulso aceptado no se reinicia con cada ventana.

Tiempos de 64 bits: evita el retorno periódico de `micros()` de 32 bits durante ensayos largos.

Frecuencia por períodos completos: permite intervalos que atraviesan las ventanas de actualización.

Conteo independiente del display: no hay deadband, EMA ni compuerta que borre volumen.

Modo seco atómico: la alarma se enclava en la ISR y los pulsos secos no se suman al volumen de líquido.

Calibración identificada: las conversiones siguen siendo provisionales mientras no se declare una calibración válida.

Una tarea propietaria: getters, `actualizar()` y control se usan desde una misma tarea; la ISR comparte únicamente estado protegido.

Sin lectura digital genérica en ISR: para el ESP32 clásico indicado se lee el registro GPIO. Los objetos deben tener almacenamiento estático en RAM interna.

El timeout de 5 s para permeado es **una elección provisional de diagnóstico**. Permite explorar períodos lentos, a costa de demorar la declaración de ausencia de pulsos; no certifica medición a bajo caudal.

caudalimetro.h

```
#pragma once

#include <Arduino.h>
#include <stdint.h>

struct ParametrosPulso {
    uint32_t anchoMinLow_us;
    uint32_t anchoMinHigh_us;
    uint32_t periodoMin_us;
    uint32_t periodoMax_us;
    float qMax_mLmin;
};

class Caudalimetro {
public:
    Caudalimetro(uint8_t pin, float k, const char* nombre,
                 bool esAlimentacion = true);

    // Llamar una sola vez, antes de iniciar los ensayos.
    bool begin();

    // Solo desde la tarea propietaria.
    void actualizar(float dt_s, bool bombaEmpuja,
                  float qBombaTeorico_mLmin = 0.0f);

    float caudal_mLmin() const { return _q; }
    float caudal_Lmin() const { return _q / 1000.0f; }
```

```

float caudal_L_min() const { return _f; }
float frecuencia_Hz() const { return _f; }
double volumen_L() const { return _vol; }

const char* nombre() const { return _nombre; }
bool esAlimentacion() const { return _esAlimentacion; }

// Diagnosticos; ausencia de pulsos no prueba ausencia de caudal.
bool sinSenal() const { return _sinSenal; }
bool falloAlimentacion() const { return _fallo; }
bool periodoDisponible() const { return _periodoDisponible; }
bool fueraDeRango() const { return _fueraRango; }
bool flujoSinImpulsion() const { return _sinImpulsion; }
bool calibrado() const { return _calibrado; }

uint32_t flancosBrutos() const { return _flancosVentana; }
uint32_t pulsosValidos() const { return _validosVentana; }
uint32_t glitchesVentana() const { return _rechazosVentana; }

// No son "pulsos reales": son ciclos que superaron el filtro.
uint64_t pulsosAceptadosAcumulados() const {
    return _aceptadosAcumulados;
}

uint64_t pulsosLiquidoAcumulados() const {
    return _liquidoAcumulados;
}

uint64_t rechazosAcumulados() const {
    return _rechazosAcumulados;
}

bool ruidoDetectadoEnSeco() const { return _ruidoEnSeco; }
bool modoSeco() const { return _modoSeco; }

void setModoSeco(bool activo);
void limpiarAlarmaSeco();

// Solo antes de begin(). Cambiar K durante un ensayo esta prohibido.
bool setK(float nuevoK);
float getK() const { return _k; }

// Solo antes de begin(); exige evidencia externa de calibracion.
bool declararCalibrado(bool valido);

private:
    static void ARDUINO_ISR_ATTR isrPuente(void* arg);
    void ARDUINO_ISR_ATTR isrInterna();

    const uint8_t _pin;
    float _k;
    const char* _nombre;
    const bool _esAlimentacion;
    ParametrosPulso _p;

    bool _iniciado = false;
    bool _calibrado = false;
    bool _modoSeco = false;

```

```

portMUX_TYPE _mux = portMUX_INITIALIZER_UNLOCKED;

// Estado compartido ISR/tarea: siempre protegido por _mux.
volatile bool _nivelAlto = false;
volatile bool _altoConocido = false;
volatile bool _candidato = false;
volatile bool _hayUltimo = false;

volatile int64_t _inicioNivel_us = 0;
volatile int64_t _bajada_us = 0;
volatile int64_t _ultimoPulso_us = 0;
volatile int64_t _ultimoPeriodo_us = 0;

volatile uint32_t _flancosISR = 0;
volatile uint32_t _aceptadosISR = 0;
volatile uint32_t _liquidoISR = 0;
volatile uint32_t _rechazosISR = 0;

volatile uint32_t _periodosISR = 0;
volatile int64_t _sumaPeriodosISR_us = 0;

volatile bool _secoISR = false;
volatile bool _alarmaSecoISR = false;

uint32_t _flancosVentana = 0;
uint32_t _validosVentana = 0;
uint32_t _rechazosVentana = 0;

uint64_t _aceptadosAcumulados = 0;
uint64_t _liquidoAcumulados = 0;
uint64_t _rechazosAcumulados = 0;

float _f = 0.0f;
float _q = 0.0f;
double _vol = 0.0;

bool _sinSenal = true;
bool _periodoDisponible = false;
bool _fueraRango = false;
bool _sinImpulsion = false;
bool _ruidoEnSeco = false;
bool _fallo = false;
float _sinPulso_s = 0.0f;
};

```

caudalimetro.cpp

```

#include "caudalimetro.h"

#include <math.h>
#include "esp_timer.h"
#include "soc/soc.h"
#include "soc/gpio_reg.h"

#if !defined(CONFIG_IDF_TARGET_ESP32)
#error "Este módulo usa el registrador GPIO del ESP32, clasico"

```

```

#error "Este modulo usa el registro GPIO del ESP32 clasico."
#endif

Caudalimetro::Caudalimetro(
    uint8_t pin, float k, const char* nombre, bool esAlimentacion)
    : _pin(pin),
      _k(k),
      _nombre(nombre),
      _esAlimentacion(esAlimentacion) {

    if (_esAlimentacion) {
        _p = {200, 200, 1000, 1000000, 1400.0f};
    } else {
        // Limites provisionales: requieren caracterizacion de banco.
        _p = {600, 600, 2500, 5000000, 200.0f};
    }
}

bool Caudalimetro::setK(float nuevoK) {
    if (_iniciado || !isfinite(nuevoK) || nuevoK <= 0.1f) {
        return false;
    }
    _k = nuevoK;
    _calibrado = false;
    return true;
}

bool Caudalimetro::declararCalibrado(bool valido) {
    if (_iniciado) return false;
    _calibrado = valido;
    return true;
}

bool Caudalimetro::begin() {
    if (_iniciado || _pin >= 32 ||
        !isfinite(_k) || _k <= 0.1f) {
        return false;
    }

    // Se usa el pull-up EXTERNO existente.
    pinMode(_pin, INPUT);

    const bool alto = (REG_READ(GPIO_IN_REG) &
                      (uint32_t(1) << _pin)) != 0;
    const int64_t ahora = esp_timer_get_time();

    portENTER_CRITICAL(&_mux);
    _nivelAlto = alto;
    _inicioNivel_us = ahora;

    // No conocemos la duracion del nivel presente al arrancar.
    // Se exige observar primero una subida.
    _altoConocido = false;
    _candidato = false;
    _hayUltimo = false;
    portEXIT_CRITICAL(&_mux);

    attachInterruptArq(pin, isrPuente, this, CHANGE);

```

```

    _iniciado = true;

    // attachInterruptArg no proporciona aqui un resultado verificable
    // de instalacion. Este true no certifica la adquisicion fisica.
    return true;
}

void Caudalimetro::setModoSeco(bool activo) {
    portENTER_CRITICAL(&_mux);
    _secoISR = activo;

    // Invalida el ciclo que cruza el cambio de modo.
    _candidato = false;
    _altoConocido = false;
    portEXIT_CRITICAL(&_mux);

    _modoSeco = activo;

    // La alarma NO se borra por repetir el comando ON.
}

void Caudalimetro::limpiarAlarmaSeco() {
    portENTER_CRITICAL(&_mux);
    _alarmaSecoISR = false;
    portEXIT_CRITICAL(&_mux);
    _ruidoEnSeco = false;
}

void ARDUINO_ISR_ATTR Caudalimetro::isrPuente(void* arg) {
    static_cast<Caudalimetro*>(arg)->isrInterna();
}

void ARDUINO_ISR_ATTR Caudalimetro::isrInterna() {
    const int64_t ahora = esp_timer_get_time();
    const bool alto = (REG_READ(GPIO_IN_REG) &
                      (uint32_t(1) << _pin)) != 0;

    portENTER_CRITICAL_ISR(&_mux);

    ++_flancosISR;
    if (_secoISR) _alarmaSecoISR = true;

    // Puede ocurrir por latencia o por eventos no resueltos.
    // Se cuenta como evento rechazado; no se inventa un flanco.
    if (alto == _nivelAlto) {
        ++_rechazosISR;
        portEXIT_CRITICAL_ISR(&_mux);
        return;
    }

    const int64_t duracion = ahora - _inicioNivel_us;

    if (!alto) {
        // Termina HIGH y comienza LOW.
        _candidato =
            _altoConocido &&
            duracion >= int64_t(_p.anchoMinHigh_us);
    }
}

```

```

    _bajada_us = ahora;

    if (!_candidato) ++_rechazosISR;
} else {
    // Termina LOW y comienza HIGH.
    if (_candidato &&
        duracion >= int64_t(_p.anchoMinLow_us)) {

        const int64_t periodo = _bajada_us - _ultimoPulso_us;

        // El control cruza ventanas de actualizar().
        if (!_hayUltimo ||
            periodo >= int64_t(_p.periodoMin_us)) {

            ++_aceptadosISR;
            if (!_secoISR) ++_liquidoISR;

            if (_hayUltimo &&
                periodo <= int64_t(_p.periodoMax_us)) {
                _ultimoPeriodo_us = periodo;
                _sumaPeriodosISR_us += periodo;
                ++_periodosISR;
            } else {
                // Primer pulso o reinicio tras una pausa larga:
                // aun no hay un periodo util.
                _ultimoPeriodo_us = 0;
            }

            _ultimoPulso_us = _bajada_us;
            _hayUltimo = true;
        } else {
            ++_rechazosISR;
        }
    } else if (_candidato) {
        ++_rechazosISR;
    }

    _candidato = false;
    _altoConocido = true;
}

_nivelAlto = alto;
_inicioNivel_us = ahora;

portEXIT_CRITICAL_ISR(&_mux);
}

void Caudalimetro::actualizar(
    float dt_s, bool bombaEmpuja, float qBombaTeorico_mLmin) {

    if (!_iniciado || !isfinite(dt_s) || dt_s <= 0.0f) {
        return;
    }

    uint32_t flancos;
    uint32_t aceptados;
    uint32_t liquido;

```

```

uint32_t liquido;
uint32_t rechazados;
uint32_t periodos;

int64_t sumaPeriodos;
int64_t ultimoPeriodo;
int64_t ultimoPulso;
int64_t ahora;
bool hayUltimo;
bool alarma;

portENTER_CRITICAL(&_mux);

flancos = _flancosISR;
aceptados = _aceptadosISR;
liquido = _liquidoISR;
rechazados = _rechazosISR;
periodos = _periodosISR;

sumaPeriodos = _sumaPeriodosISR_us;
ultimoPeriodo = _ultimoPeriodo_us;
ultimoPulso = _ultimoPulso_us;
hayUltimo = _hayUltimo;
alarma = _alarmaSecoISR;

ahora = esp_timer_get_time();

_flancosISR = 0;
_aceptadosISR = 0;
_liquidoISR = 0;
_rechazosISR = 0;
_periodosISR = 0;
_sumaPeriodosISR_us = 0;

portEXIT_CRITICAL(&_mux);

_flancosVentana = flancos;
_validosVentana = aceptados;
_rechazosVentana = rechazados;

_aceptadosAcumulados += aceptados;
_liquidoAcumulados += liquido;
_rechazosAcumulados += rechazados;
_ruidoEnSeco = alarma;

// Conversion provisional si calibrado() es false.
// No se elimina volumen por deadband o por estado de la bomba.
_vol += double(liquido) / (60.0 * double(_k));

_sinSenal =
    !hayUltimo ||
    (ahora - ultimoPulso) > int64_t(_p.periodoMax_us);

_periodoDisponible = !_sinSenal && ultimoPeriodo > 0;

if (_sinSenal) {
    _f = 0.0f;
    _q = 0.0f;
}

```

```

} else if (!_periodoDisponible) {
    // Hay un pulso, pero aun no puede estimarse una frecuencia.
    _f = NAN;
    _q = NAN;
} else {
    if (periodos > 0 && sumaPeriodos > 0) {
        // Media temporal sobre periodos completos, no sobre
        // la duracion arbitraria de la ventana del loop.
        _f = float(1.0e6 * double(periodos) /
                    double(sumaPeriodos));
    } else {
        // Retencion entre pulsos, limitada por periodoMax_us.
        _f = float(1.0e6 / double(ultimoPeriodo));
    }

    _q = 1000.0f * _f / _k;

    _fueraRango = isfinite(_q) && _q > _p.qMax_mLmin;

    _sinImpulsion =
        !_esAlimentacion &&
        (!bombaEmpuja ||
         (isfinite(qBombaTeorico_mLmin) &&
          qBombaTeorico_mLmin < 20.0f)) &&
        ((aceptados > 0) || (isfinite(_q) && _q > 0.0f));

    if (aceptados > 0 || !bombaEmpuja || !_esAlimentacion) {
        _sinPulso_s = 0.0f;
        _fallo = false;
    } else {
        _sinPulso_s += dt_s;
        _fallo = _sinPulso_s >= 5.0f;
    }
}
}

```

Límite importante: `glitchesVentana()` ahora representa eventos rechazados por la lógica, no una medición del número físico de perturbaciones EMI. Un episodio puede producir varios eventos, y algunos flancos pueden no observarse.

El módulo tampoco conserva compatibilidad con `resetVolumen()` ni con el alias engañoso `pulsosBrutos()`. Para sesiones deben usarse diferencias de contadores acumulados, sin reiniciar el historial de adquisición.

5.2. Ajustes de `config.h`

Conservar los demás parámetros existentes, pero añadir comprobaciones de colisión y declarar explícitamente el estado provisional de la calibración:

```
// Estas comprobaciones NO verifican el cableado físico
```

```
// Estas comprobaciones NO verifican el cableado físico.
static_assert(
    PIN_SENSOR_ALIMENTACION != PIN_SENSOR_PERMEADO,
    "Los sensores no pueden compartir GPIO");

static_assert(
    PIN_SENSOR_ALIMENTACION != PIN_PUL &&
    PIN_SENSOR_ALIMENTACION != PIN_DIR &&
    PIN_SENSOR_ALIMENTACION != PIN_ENA &&
    PIN_SENSOR_PERMEADO != PIN_PUL &&
    PIN_SENSOR_PERMEADO != PIN_DIR &&
    PIN_SENSOR_PERMEADO != PIN_ENA,
    "Colision entre sensores y salidas del motor");

// Cambiar a true SOLO tras documentar la calibracion correspondiente.
constexpr bool CAL_ALIMENTACION_VALIDADA = false;
constexpr bool CAL_PERMEADO_VALIDADA = false;

// Los K existentes se conservan solamente para comparacion
provisional.
// No reemplazarlos por 98 sin medir.
```

No cambio automáticamente los GPIO ni los K, porque hacerlo elegiría una de dos descripciones incompatibles sin evidencia.

En `setup()`, antes de `begin()` :

```
sensorAlimentacion.declararCalibrado(CAL_ALIMENTACION_VALIDADA);
sensorPermeado.declararCalibrado(CAL_PERMEADO_VALIDADA);

const bool inicioAlim = sensorAlimentacion.begin();
const bool inicioPerm = sensorPermeado.begin();

if (!inicioAlim || !inicioPerm) {
    Serial.println("ERROR: inicializacion de adquisicion");
    // La aplicacion debe impedir habilitar la bomba.
}
```

5.3. Registro sin sobrescritura: `registro_ensayos.h`

Este módulo completo conserva **600 filas totales y hasta 20 sesiones**, sin borrar las anteriores. El inicio y el fin consumen filas. Se usa RAM: no promete conservar datos tras un reinicio ni reemplaza la exportación o almacenamiento persistente.

Registra los valores de la última actualización junto con su instante real de adquisición; así no confunde la hora del registro con la edad de la medición.

```
#pragma once

#include <Arduino.h>
#include <math.h>
#include "esp_timer.h"
#include "caudalimetro.h"

class RegistroEnsayos {
```

```

public:
    enum Tipo : uint8_t {
        INICIO = 0,
        PERIODICA = 1,
        FIN = 2
    };

    enum Calidad : uint16_t {
        CAL_A = 1u << 0,
        CAL_P = 1u << 1,
        PERIODO_A = 1u << 2,
        PERIODO_P = 1u << 3,
        SIN_SENAL_A = 1u << 4,
        SIN_SENAL_P = 1u << 5,
        FUERA_RANGO_A = 1u << 6,
        FUERA_RANGO_P = 1u << 7,
        SECO_A = 1u << 8,
        SECO_P = 1u << 9,
        RUIDO_SECO_A = 1u << 10,
        RUIDO_SECO_P = 1u << 11,
        SIN_IMPULSION_P = 1u << 12
    };

    struct Fila {
        uint16_t sesion;
        uint8_t tipo;
        uint16_t calidad;

        int64_t tRegistro_us;
        int64_t tAdquisicion_us;

        uint64_t nA;
        uint64_t nP;
        uint64_t rechazosA;
        uint64_t rechazosP;

        float rpm;
        float qA;
        float qP;
        float kA;
        float kP;
    };

    static constexpr size_t CAPACIDAD = 600;
    static constexpr uint16_t SESIONES_MAX = 20;
    static constexpr int64_t INTERVALO_US = 10000000;

    // Debe ser un objeto global/estatico, no una variable local grande.
    Fila filas[CAPACIDAD];

    size_t cantidad() const { return _n; }
    bool activo() const { return _activo; }
    bool incompleto() const { return _incompleto; }
    uint32_t intervalosOmitidos() const { return _omitidos; }

    bool iniciar(int64_t tAdquisicion_us, float rpm,
                const Caudalimetro& a, const Caudalimetro& p) {
        if ( _activo || _sesion >= SESIONES_MAX) return false;

```

```

if (!_activo || _sesion > SESIONES_MAX) return false;

// Reserva al menos inicio y fin.
if (_n + 2 > CAPACIDAD) {
    _incompleto = true;
    return false;
}

++_sesion;
_inicio_us = esp_timer_get_time();
_proximo_us = _inicio_us + INTERVALO_US;
_activo = true;

guardar(INICIO, _inicio_us, tAdquisicion_us, rpm, a, p);
return true;
}

void tick(int64_t tAdquisicion_us, float rpm,
          const Caudalimetro& a, const Caudalimetro& p) {
    if (!_activo) return;

    const int64_t ahora = esp_timer_get_time();
    if (ahora < _proximo_us) return;

    const uint64_t vencidos =
        uint64_t((ahora - _proximo_us) / INTERVALO_US) + 1;

    _proximo_us += int64_t(vencidos) * INTERVALO_US;

    // No se fabrican filas retrospectivas con datos repetidos.
    if (vencidos > 1) {
        _omitidos += uint32_t(vencidos - 1);
        _incompleto = true;
    }

    // Conserva una plaza para el FIN.
    if (_n >= CAPACIDAD - 1) {
        _omitidos++;
        _incompleto = true;
        return;
    }

    guardar(PERIODICA, ahora, tAdquisicion_us, rpm, a, p);
}

bool finalizar(int64_t tAdquisicion_us, float rpm,
               const Caudalimetro& a, const Caudalimetro& p) {
    if (!_activo) return false;

    const bool ok = guardar(
        FIN, esp_timer_get_time(), tAdquisicion_us, rpm, a, p);

    _activo = false;
    return ok;
}

private:
    size_t _n = 0;

```

```

uint16_t _sesion = 0;
bool _activo = false;
bool _incompleto = false;
uint32_t _omitidos = 0;

int64_t _inicio_us = 0;
int64_t _proximo_us = 0;

static uint16_t calidad(
    const Caudalimetro& a, const Caudalimetro& p) {
    uint16_t f = 0;

    if (a.calibrado()) f |= CAL_A;
    if (p.calibrado()) f |= CAL_P;
    if (a.periodoDisponible()) f |= PERIODO_A;
    if (p.periodoDisponible()) f |= PERIODO_P;
    if (a.sinSenal()) f |= SIN_SENAL_A;
    if (p.sinSenal()) f |= SIN_SENAL_P;
    if (a.fueraDeRango()) f |= FUERA_RANGO_A;
    if (p.fueraDeRango()) f |= FUERA_RANGO_P;
    if (a.modoSeco()) f |= SECO_A;
    if (p.modoSeco()) f |= SECO_P;
    if (a.ruidoDetectadoEnSeco()) f |= RUIDO_SECO_A;
    if (p.ruidoDetectadoEnSeco()) f |= RUIDO_SECO_P;
    if (p.flujoSinImpulsion()) f |= SIN_IMPULSION_P;

    return f;
}

bool guardar(
    Tipo tipo, int64_t ahora, int64_t adquisicion,
    float rpm, const Caudalimetro& a, const Caudalimetro& p) {

    if (_n >= CAPACIDAD) {
        _incompleto = true;
        return false;
    }

    Fila& r = filas[_n++];
    r.sesion = _sesion;
    r.tipo = uint8_t(tipo);
    r.calidad = calidad(a, p);

    r.tRegistro_us = ahora - _inicio_us;

    // -1 significa que no hubo adquisicion desde el inicio.
    r.tAdquisicion_us =
        adquisicion >= _inicio_us ? adquisicion - _inicio_us : -1;

    r.nA = a.pulsosLiquidoAcumulados();
    r.nP = p.pulsosLiquidoAcumulados();
    r.rechazosA = a.rechazosAcumulados();
    r.rechazosP = p.rechazosAcumulados();

    r.rpm = rpm;

    // Inicio: evento, no medicion de una ventana inexistente.
    return true;
}

```

```

const bool tieneMedicion =
    tipo != INICIO && r.tAdquisicion_us >= 0;

r.qA = tieneMedicion ? a.caudal_mLmin() : NAN;
r.qP = tieneMedicion ? p.caudal_mLmin() : NAN;

r.kA = a.getK();
r.kP = p.getK();

return true;
}
};

```

Interpretación: los contadores son acumulados desde el arranque y actualizados por `actualizar()`. Para calcular caudales medios deben restarse filas con tiempos de adquisición válidos y distintos; no usar la fila `INICIO` como si definiera una ventana de conteo exactamente sincronizada.

Si se requiere un volumen estrictamente comprendido entre los comandos START y STOP, hará falta agregar una captura atómica específica de frontera de sesión. El módulo anterior no finge tener esa precisión: registra el instante de las capturas disponibles.

5.4. Integración en el `.ino`

Declarar globalmente:

```

#include "registro_ensayos.h"

RegistroEnsayos registro;
int64_t ultimaAdquisicion_us = -1;

```

En el bloque de adquisición, inmediatamente después de actualizar ambos sensores:

```

ultimaAdquisicion_us = esp_timer_get_time();

```

```

const float qAlim = sensorAlimentacion.caudal_mLmin();
const float qPerm = sensorPermeado.caudal_mLmin();

const bool conversionesDisponibles =
    isfinite(qAlim) && isfinite(qPerm);

// No confundir disponibilidad numerica con validez metrologica.
const bool calibracionDisponible =
    sensorAlimentacion.calibrado() &&
    sensorPermeado.calibrado();

qRet_mLmin =
    conversionesDisponibles ? qAlim - qPerm : NAN;

recuperacion =
    conversionesDisponibles && qAlim > 0.0f
        ? 100.0f * qPerm / qAlim
        : NAN;

jLMH_actual =
    isfinite(qPerm)
        ? qPerm * 0.06f / AREA_MEMBRANA_M2
        : NAN;

// Diagnostico, no correccion de las mediciones.
flagCruceSensores =
    conversionesDisponibles && qPerm > qAlim + 50.0f;

// Mantener Darcy invalido.
// El serializador debe exportar presiones no medidas como null/vacio.
resultadoDarcy = ResultadoDarcy{};

// Publicar calibracionDisponible y las banderas individuales en SCADA.
(void)calibracionDisponible;

```

En las transiciones de ensayo:

```

if (enMarcha && !bombaEnMarchaAnterior) {

```

```

const bool registrado = registro.iniciar(
    ultimaAdquisicion_us,
    bomba.rpmActual(),
    sensorAlimentacion,
    sensorPermeado);

if (!registrado) {
    Serial.println("ENSAYO SIN REGISTRO: capacidad o estado invalido");
    // La politica de operacion debe impedir continuar
    inadvertidamente.
}

if (!enMarcha && bombaEnMarchaAnterior) {
    registro.finalizar(
        ultimaAdquisicion_us,
        bomba.rpmActual(),
        sensorAlimentacion,
        sensorPermeado);
}

```

Y sustituir el bloque anterior de datalogger por:

```

registro.tick(
    ultimaAdquisicion_us,
    bomba.rpmActual(),
    sensorAlimentacion,
    sensorPermeado);

```

No mantener simultáneamente ambos registradores sin decidir cuál es la fuente oficial. El exportador CSV existente debe adaptarse a estas filas, incluir `tipo`, tiempos, K y banderas, y convertir los valores no finitos en campos vacíos.

El criterio `bomba.enMarcha()` también debe revisarse: sin su implementación no puede saberse si FIN significa comando de parada, final de rampa o detención efectiva. Para estudiar descarga residual, la adquisición y el registro deben continuar después del comando STOP.

6. Pruebas necesarias para levantar el rechazo

La aprobación debe apoyarse en **criterios de aceptación acordados antes del ensayo**, no en que el SCADA parezca estable.

Correspondencia de pines y señal eléctrica. Verificar GPIO reales, alimentación y tipo de salida del sensor. Comprobar que el acondicionamiento entregue niveles compatibles con el

salida del sensor. Comprobar que el acondicionamiento entregue niveles compatibles con el ESP32; las fichas comerciales no prueban que la unidad instalada sea realmente open-collector.

Pruebas digitales del algoritmo. Inyectar pulsos conocidos, incluidos 200 y 600 μ s en sus fronteras, pulsos lentos, interrupciones de señal, ráfagas y eventos a ambos lados del corte de actualización. Comparar conteos enviados y aceptados.

Cero hidráulico con motor activo. Evaluar alimentación y permeado sin flujo real, incluyendo rampas y todas las RPM previstas. Registrar tanto flancos rechazados como pulsos aceptados. La aparición de cualquier pulso aceptado impide declarar éxito para esa condición.

Ensayo húmedo de conservación de pulsos. Confirmar que el filtro no suprime señal real cuando aparecen rebotes. Medir a ambos lados del acondicionamiento si es accesible, sin retirar componentes y con instrumentación adecuada.

Calibración individual y repetida. Recolectar volumen o masa de referencia con tiempo registrado. Evaluar puntos ascendentes y descendentes, especialmente 10, 25, 50, 100 y 200 mL/min en permeado, sin asumir que todos resultarán medibles.

Ensayo de carga informática. Ejecutar WiFi, consultas SCADA y exportación mientras se compara el conteo con una referencia. Probar pausas del servidor, operaciones de flash si existen y duración prolongada.

Integridad del registro. Verificar inicio, fin, múltiples sesiones, capacidad agotada, exportación y comportamiento ante reinicio. Confirmar que no se sobrescriban datos ni se rellenen tiempos perdidos con valores ficticios.

Presupuesto de incertidumbre. Incorporar referencia volumétrica o gravimétrica, cronometraje, cuantización, repetibilidad, dependencia con caudal, pulsación y pérdidas del sistema de adquisición. Definir el rango final únicamente donde el desempeño resulte aceptable.

Para completar la revisión de integración faltan el proyecto íntegro y el mapa de conexiones verificado, especialmente la resolución de GPIO 18/19 frente a 14/27. Con la información actual queda rechazada la certificación V5.0; queda justificada una nueva etapa de validación por software con el hardware sin modificar.